

AI-POWERED CROP DISEASE DETECTION FOR FOOD SECURITY

A

MINOR PROJECT-I REPORT

Submitted in partial fulfilment of the requirements

for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

By

GROUP NO. 27

Shivam Kumar	0187CS231217
Shrit Kumar	0187CS231218
Vivek Kumar	0187CS231257
Yash Raj	0187CS231260

Under the guidance of

Dr. Komal Tahiliani

(Associate Professor)



Department of Computer Science & Engineering
Sagar Institute of Science & Technology (SISTec), Bhopal (M.P)
Approved by AICTE, New Delhi & Govt. of M.P.
Affiliated to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.)

December – 2025

Sagar Institute of Science & Technology (SISTec), Bhopal (M.P.)
Department of Computer Science & Engineering



CERTIFICATE

We hereby certify that the work which is being presented in the B.Tech. Minor Project-I Report entitled **AI-Powered Crop Disease Detection for Food Security**, in partial fulfilment of the requirements for the award of the degree of ***Bachelor of Technology***, submitted to the Department of **Computer Science & Engineering**, Sagar Institute of Science & Technology (SISTec), Bhopal (M.P.) is an authentic record of our own work carried out during the period from Jul-2025 to Dec-2025 under the supervision of **Dr. Komal Tahiliani**.

The content presented in this project has not been submitted by me for the award of any other degree elsewhere.

Shivam Kumar 0187CS231217	Shrit Kumar 0187CS231218	Vivek Kumar 0187CS231257	Yash Raj 0187CS231260
--	---	---	--

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Dr Komal Tahiliani
Project Guide

Prof. Nargish Gupta
HOD, CSE

Dr. Manish Billore
Principal

ACKNOWLEDGEMENT

We would like to express our sincere thanks **Dr. Manish Billore, Principal, SISTec** and **Dr. Swati Saxena, Vice Principal SISTec** Gandhi Nagar, Bhopal for giving us an opportunity to undertake this project.

We also take this opportunity to express a deep sense of gratitude to **Prof. Nargish Gupta, HOD, Department of Computer Science & Engineering** for his kindhearted support

We extend our sincere and heartfelt thanks to our guide, **Dr. Komal Tahiliani** , for providing us with the right guidance and advice at crucial junctures and for showing us the right way.

I am thankful to the **Project Coordinator, Prof. Mayank Kurchaniya**, who devoted his precious time in giving us the information about various aspects and gave support and guidance at every point of time.

I would like to thank all those people who helped me directly or indirectly to complete my project whenever I found myself in any issue.

TABLE OF CONTENTS

TITLE	PAGE NO.
Abstract	i
List of Abbreviation	ii
List of Figures	iii
Chapter 1	Introduction
	1
1.1	About Project
	1
1.2	Project Objectives
	1
Chapter 2	Software & Hardware Requirements
	3
2.1	Software Requirements
	3
2.2	Hardware Requirements
	4
Chapter 3	Problem Description
	5
3.1	Introduction
	5
3.2	Problem Statement
	5
3.3	Objectives of the Project
	5
3.4	Scope of the Project
	6
3.5	Challenges & Constraints
	6
3.6	Expected Outcomes
	7
Chapter 4	Literature Survey
	8
Chapter 5	Software Requirements Specification
	10
5.1	Functional Requirements
	10
5.2	Non-Functional Requirements
	11
Chapter 6	Software and Hardware Design
	12
6.1	Use Case Diagram
	12
6.2	Architecture
	13
Chapter 7	ML Module
	15
7.1	Introduction
	15
7.2	Problem Definition and Algorithm
	15
7.3	ML Model Description
	15
7.4	Result Analysis
	16
Chapter 8	Coding
	17
8.1	Code Implementation
	17

Chapter 9	Result & Output Screens	33
	9.1 Results	33
	9.2 Output Screens	33
Chapter 10	Conclusions & Future Work	35
	10.1 Conclusion	35
	10.2 Future Work	35
References		
Project Summary		
Appendix1: Glossary of Terms		

ABSTRACT

Agriculture is the backbone of the economy, and crop diseases are a major cause of reduced agricultural productivity and financial losses for farmers. Early identification and proper treatment of plant diseases are essential for improving crop yield and ensuring food security; however, traditional disease detection methods rely on manual observation and expert consultation, which are time-consuming, costly, and not easily accessible to all farmers. The AI-Powered Crop Disease Detection for Food Security is a web-based application that uses Artificial Intelligence and image processing techniques to identify crop diseases accurately by analysing uploaded leaf images. In addition to disease detection, the system provides detailed diagnostic information such as symptoms, confidence score, treatment recommendations, and preventive measures, along with real-time weather integration to support better agricultural decision-making. The platform also includes secure user authentication and maintains a history of previous detections for long-term crop monitoring. Developed using modern web technologies such as React, Flask, and Firebase, this system promotes smart and sustainable agriculture by reducing crop losses, improving decision-making, and supporting digital transformation in the agricultural sector.

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
AI	Artificial Intelligence
SDLC	Software Development Life Cycle
HTML	Hyper Text Markup Language
UML	Unified Modelling Language

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 6.1	: Use Case diagram	13
Figure 9.2.1	: User Login page	32
Figure 9.2.2	: User Signup Page	33
Figure 9.2.3	: Home Page	33
Figure 9.2.4	: Dashboard page	34
Figure 9.2.5	: Disease Detection Page	00
Figure 9.2.6	: Detection Reports / History Page	00
Figure 9.2.7	: Govt. Schemes Page	00

Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

1.1 ABOUT PROJECT

Agriculture plays a vital role in sustaining human life and economic development. One of the biggest challenges faced by farmers is the timely identification of crop diseases, which can significantly affect crop quality and yield if not treated at an early stage. In many rural and semi-urban areas, access to agricultural experts and diagnostic facilities is limited, making disease detection difficult and delayed.

The **AI-Powered Crop Disease Detection for Food Security** is developed to address these challenges by providing an intelligent, automated, and user-friendly solution for crop disease identification. This system enables farmers and gardeners to upload images of plant leaves through a web-based interface. The uploaded images are processed using Artificial Intelligence and Machine Learning models to detect diseases accurately.

The system not only identifies diseases but also provides comprehensive diagnostic information, including symptoms, treatment methods, and preventive measures. Additionally, real-time weather information is integrated into the platform to assist users in agricultural planning. A detection history feature allows users to track previous analyses, supporting long-term crop health monitoring.

By digitizing the crop disease detection process, this project reduces dependency on manual inspection, minimizes crop losses, and promotes sustainable farming practices. The system aligns with modern digital agriculture initiatives and supports the use of technology to enhance productivity and efficiency in the agricultural sector.

1.2 PROJECT OBJECTIVES

The main objective of the **AI-Powered Crop Disease Detection for Food Security** is to develop an intelligent and reliable platform that assists users in early detection and management of crop diseases. The specific objectives of the project are as follows:

1.2.1 Automated Disease Detection

To detect crop diseases automatically using AI-based image analysis techniques.

1.2.2 Accurate Diagnosis

To provide accurate disease identification along with confidence scores and detailed diagnostic information.

1.2.3 Treatment and Prevention Guidance

To suggest suitable treatment methods and preventive measures for detected crop diseases.

1.2.4 Weather-Based Agricultural Support

To integrate real-time weather information to assist farmers in planning agricultural activities.

1.2.5 Detection History Management

To maintain a record of previous disease detections for future reference and monitoring.

1.2.6 User-Friendly Interface

To develop a simple, responsive, and easy-to-use web interface accessible on both desktop and mobile devices.

1.2.7 Secure Data Handling

To ensure secure user authentication and safe storage of user data and detection records.

1.2.8 Support Sustainable Agriculture

To reduce crop losses and promote smart and sustainable farming through digital technology.

Chapter 2

Software and Hardware Requirements

CHAPTER 2

SOFTWARE & HARDWARE REQUIREMENTS

2.1 SOFTWARE REQUIREMENTS

The software components used in the application are essential for analyzing crop images, detecting plant diseases accurately, and providing an interactive and user-friendly platform for farmers and agricultural users.

The specific requirements are as follows:

2.1.1 Development Tools

Frontend Development: React (Vite), JavaScript

Styling Framework: Tailwind CSS

IDE: Visual Studio Code

Backend Development: Python (Flask)

API Testing Tool: Postman

2.1.2 APIs

Weather API for real-time weather information

Google Search API for supplementary plant information

2.1.3 User Requirements

Modern Web Browser

Operating System: Windows 10/11, Linux, MacOS

2.1.4 Database (Firebase Firestore)

Firebase Firestore is used to store dynamic data such as user profiles, crop disease detection records, weather details, and detection history. Its cloud-based scalability and real-time synchronization support efficient data management and fast access for agricultural users.

2.2 HARDWARE REQUIREMENTS

The hardware components ensure smooth functioning, development, and usage of the application. The specific requirements are as follows:

2.2.1 Developer Machines

Processor: Quad-core or higher

RAM: Minimum 4 GB

Storage: Minimum 256 GB

Operating System: Windows 10/11, Linux, or macOS

2.2.2 Test Devices

PC / Laptop

Mobile Phones

Web Browsers

2.2.3 User Requirements

Device: Smartphone or Tablet (Android / iOS) or PC

Processor: Dual-core or higher

RAM: 2 GB or higher

Camera: Required for capturing crop images

Chapter 3

Problem

Description

CHAPTER 3

PROBLEM DESCRIPTION

3.1 INTRODUCTION

In the agricultural sector, crop diseases remain one of the major causes of reduced crop yield and economic loss for farmers. Traditional methods of disease detection largely depend on manual observation, expert consultation, and laboratory testing, which are time-consuming, costly, and not easily accessible, especially in rural areas. Due to the lack of a centralized and automated disease detection system, identifying plant diseases at an early stage becomes difficult, leading to delayed treatment and increased crop damage.

Moreover, farmers often find it challenging to maintain proper records of crop health, disease history, and environmental conditions, while agricultural experts face difficulties in providing timely support to a large number of farmers. These limitations restrict effective decision-making related to disease control, crop management, and resource utilization.

There is a strong need for a reliable, secure, and real-time digital system that can automatically detect crop diseases using modern technologies. Such a system must enable users to upload crop images easily and receive accurate diagnostic information along with treatment recommendations. This project addresses these challenges by developing a scalable, AI-based, and user-friendly web application aimed at improving agricultural productivity, disease management, and sustainable farming practices.

3.2 PROBLEM STATEMENT

The traditional crop disease identification process is largely manual, time-consuming, and dependent on expert knowledge, which is not always readily available. This project aims to develop a secure and intelligent web-based system that automates crop disease detection using artificial intelligence. The system will allow users to upload images of crop leaves and receive instant disease identification results along with symptoms, treatment suggestions, and preventive measures. It will also provide real-time weather information and maintain a history of previous detections. By streamlining disease diagnosis and providing timely information, the system supports better agricultural decision-making, reduces crop loss, and promotes smart farming practices.

3.3 OBJECTIVES OF THE PROJECT

The objectives of the **AI-Powered Crop Disease Detection for Food Security** aim to address the above problems by providing an intelligent, efficient, and farmer-friendly solution. The objectives of the project are:

- To eliminate traditional manual methods of crop disease identification by introducing an AI-based automated system.
- To allow users to upload crop images and receive accurate disease detection results in real-time.
- To provide detailed disease information including symptoms, treatment methods, and preventive measures.
- To store disease detection data securely in a centralized database for future reference and monitoring.
- To integrate real-time weather information to support agricultural planning and disease prevention.
- To develop a simple and user-friendly interface accessible to farmers and agricultural users.

3.4 SCOPE OF THE PROJECT

The project aims to develop a secure, web-based platform for automated crop disease detection using artificial intelligence. It allows users to upload images of crop leaves and receive instant diagnostic results. The system provides disease details, treatment recommendations, and preventive suggestions along with weather-based agricultural insights. A detection history feature enables long-term monitoring of crop health. The platform is scalable and can be extended to support multiple crop varieties, making it suitable for modern digital agriculture and sustainable farming initiatives.

3.5 CHALLENGES AND CONSTRAINTS

Despite the advantages of developing an AI-based crop disease detection system, the project may face the following challenges and constraints:

- **Image Quality:** Poor-quality or unclear images may affect the accuracy of disease detection.
- **Data Accuracy:** The effectiveness of AI models depends on the quality and diversity of training data.
- **Internet Dependency:** The system requires a stable internet connection for image upload and real-time analysis.
- **User Adoption:** Farmers unfamiliar with digital tools may require guidance and training.
- **System Scalability:** Handling a large number of users and image uploads may impact performance if not optimized.

3.6 EXPECTED OUTCOMES

The project is expected to deliver a reliable and user-friendly AI-based web application that simplifies crop disease detection. Farmers and agricultural users will be able to identify diseases early, receive accurate treatment guidance, and access weather-based insights for better crop management. The system will reduce dependency on manual diagnosis, minimize crop losses, and ensure secure handling of user data. Overall, it will promote efficiency, data-driven agricultural decisions, and support the vision of smart and sustainable farming.

Chapter 4

Literature Survey

CHAPTER 4

LITERATURE SURVEY

Studies in the domain of **smart agriculture and crop disease management** emphasize the need for technology-driven and automated disease detection mechanisms. Traditional methods of identifying crop diseases rely heavily on manual inspection, expert consultation, and visual observation, which often lead to delays, inaccuracies, and inconsistent diagnosis. Literature indicates that AI-based disease detection systems provide more reliable results through automated image analysis, centralized data storage, and standardized classification techniques. Such systems enable farmers to identify plant diseases at an early stage, monitor crop health efficiently, and take timely preventive actions. These findings highlight the importance of adopting an intelligent digital solution that minimizes manual dependency, improves diagnostic accuracy, and streamlines crop disease management processes for sustainable agriculture.[1]

Research in **agricultural information systems** highlights the importance of secure, web-based platforms for managing crop health data and user records. Studies point out that agricultural applications often handle sensitive information such as user profiles, crop images, disease reports, and location-based data. Without adequate security measures, these systems may become vulnerable to unauthorized access and data misuse. Literature consistently recommends the implementation of secure authentication mechanisms, encrypted data storage, and controlled access policies to ensure data protection. Additionally, role-based interfaces and personalized dashboards significantly enhance usability by presenting relevant information to farmers and agricultural experts. These insights reinforce the necessity of adopting secure authentication technologies and access control mechanisms while building a robust crop disease detection system.[2]

Recent discussions on **digital transformation in agriculture** highlight the increasing adoption of web and mobile platforms that enable seamless interaction between farmers, technology providers, and agricultural services. Literature emphasizes the role of modern user interfaces that focus on simplicity, responsiveness, and quick access to diagnostic results through API-based communication. Such platforms allow users to upload crop images, receive instant disease analysis, and access treatment recommendations with minimal technical complexity. Research also underlines the advantages of modular UI components and reusable design patterns, which improve system consistency and reduce development complexity. Technologies such as React, Tailwind CSS, and Axios support these design principles by enabling responsive layouts, efficient styling, and smooth client-server communication, making them suitable for developing intelligent agricultural applications.[3]

Modern scholarly reviews on **precision agriculture and AI-driven farming solutions** highlight a growing shift toward automated, data-driven crop management systems. These systems provide real-

time insights into crop health, generate actionable recommendations, and maintain historical disease records across multiple farming cycles. Literature shows that farmers benefit from structured dashboards that display detection history, weather conditions, and treatment guidance, while agricultural planners benefit from aggregated data for trend analysis and decision-making. Many digital agriculture platforms discussed in the literature adopt cloud-based architectures, scalable components, and secure workflows to ensure long-term reliability and accessibility. These findings closely align with the objectives of the proposed **AI-Powered Crop Disease Detection for Food Security**, which aims to simplify disease diagnosis, enhance productivity, and promote sustainable farming through intelligent technology integration.[4]

Chapter 5

Software

Requirements

Specification

CHAPTER 5

SOFTWARE REQUIREMENTS SPECIFICATION

5.1 FUNCTIONAL REQUIREMENTS

The functional requirements of the application outline the core functionalities the system must perform. These requirements define the operations the system is expected to execute, ensuring it provides a seamless, efficient, and user-centric experience that fulfills its intended purpose of detecting crop diseases, providing diagnosis details, and supporting agricultural decision-making.

5.1.1 USER REGISTRATION AND AUTHENTICATION

Users must be able to sign up and log in using email and password. Only authenticated users can access disease detection features, view diagnosis results, and manage detection history. Users must also be able to log out securely from the system.

5.1.2 CROP IMAGE UPLOAD AND DISEASE DETECTION

Users can upload crop or leaf images for analysis. The system processes the uploaded images using AI models to identify plant diseases. The system should return disease name, confidence score, symptoms, treatment recommendations, and preventive measures.

5.1.3 WEATHER INFORMATION AND ADVISORY

The system shall fetch and display real-time weather information based on the user's location. Weather details should assist users in planning agricultural activities and preventing disease spread under unfavorable environmental conditions.

5.1.4 DETECTION HISTORY MANAGEMENT

The system must store disease detection records for each user, including image details, detected disease, and date. Users should be able to view and manage previous detection results for monitoring crop health over time.

5.1.5 USER EXPERIENCE

The user interface must be designed with a strong focus on user experience (UX). It should be simple, intuitive, and easy to navigate, allowing farmers and agricultural users to interact smoothly with all system features.

These functional requirements ensure that the application is comprehensive, user-friendly, and effective in delivering accurate disease detection and agricultural insights.

5.1.6 ACTIVITY LOGGING

The system shall automatically record user activities such as “Image Uploaded”, “Disease Detected”, and “Weather Data Viewed” to support tracking and analysis.

5.2 NON-FUNCTIONAL REQUIREMENTS

The AI-Powered Crop Disease Detection for Food Security is designed to meet several key non-functional requirements. These requirements are essential to ensure that the system performs efficiently, securely, and reliably under real-world agricultural usage conditions.

5.2.1 PERFORMANCE

The performance of the AI-Powered Crop Disease Detection for Food Security is a critical non-functional requirement that directly affects user satisfaction and usability. The system is designed to ensure fast response times during image uploads, disease analysis, and result retrieval, even under high user load. It should efficiently handle concurrent requests without delays or timeouts.

5.2.2 RELIABILITY

Reliability ensures the system operates consistently with minimal downtime. The AI-Powered Crop Disease Detection for Food Security will include proper error handling, stable API integration, and recovery mechanisms to prevent failures and ensure continuous availability of services.

5.2.3 USABILITY

Usability focuses on providing an intuitive and user-friendly interface that allows farmers and agricultural users to use the system without technical difficulty. The system should require minimal training and offer clear instructions for all features.

5.2.4 SECURITY

Security ensures the protection of sensitive user data, crop images, and detection records through secure authentication, encrypted communication, and strict access control. The system will safeguard user privacy and prevent unauthorized access.

5.2.5 MAINTAINABILITY

Maintainability ensures the system can be easily updated, modified, and enhanced in the future. The application will follow a modular architecture and clear documentation to simplify debugging, maintenance, and future improvements.

Chapter 6

Software And

Hardware Design

CHAPTER 6

SOFTWARE AND HARDWARE DESIGN

6.1 USE CASE DIAGRAM

The use case diagram for the **AI-Powered Crop Disease Detection for Food Security** demonstrates the interaction between various actors and the system, highlighting the key functionalities and how they connect to external services. The diagram provides a clear overview of user interactions with the system and its major operational features.

Key Elements

Actors represent users or external entities interacting with the system.

Use Cases denote the actions or services performed by the system for the users.

Description

The primary actor in this system is the **Farmer / Agricultural User**, who interacts with the web-based platform to identify crop diseases and monitor crop health. Users can register, log in securely, upload crop or leaf images, and receive disease diagnosis results along with treatment and prevention recommendations. Users can also view real-time weather information and manage their disease detection history for long-term crop monitoring.

The **Administrator** accesses the Admin Panel to manage user accounts, monitor system activities, and review detection reports. Admins can view logs, analyze usage patterns, and ensure the smooth functioning of the system. This interaction ensures reliable system management and supports efficient agricultural decision-making.

Below is the Use-Case diagram:-

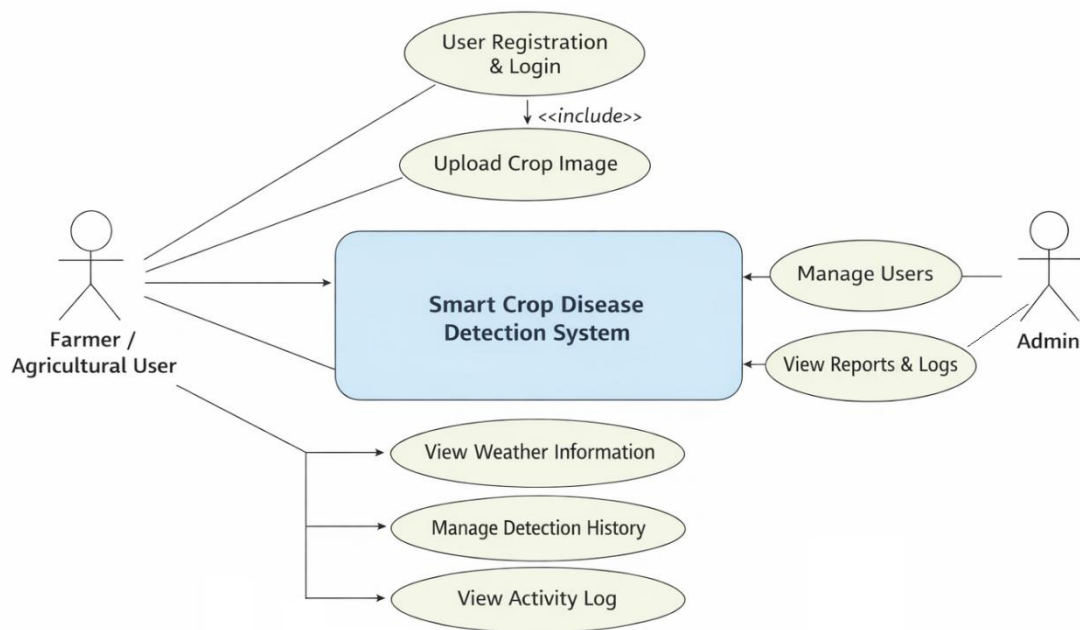


Figure 6.1: Use Case Diagram

6.2 ARCHITECTURE

The architecture of the **AI-Powered Crop Disease Detection for Food Security** is designed using a three-tier architecture to ensure scalability, maintainability, and clarity. The overall architecture separates presentation, application logic, and data management layers.

Client(Frontend)

The frontend provides a user-friendly interface for user registration, login, image upload, and result visualization. It supports input validation and smooth interaction, enabling users to upload crop images, view disease diagnosis, access weather data, and manage detection history efficiently.

Backend

The backend handles API requests, processes uploaded images, and communicates with AI-based disease detection services. It manages secure user authentication, role-based access control, activity logging, and integration with external APIs to ensure accurate and secure data processing.

Database

The database stores structured data such as user profiles, disease detection records, weather details, and activity logs. Cloud-based storage is used for scalability and real-time data synchronization.

APIs

The system interacts with AI-based APIs for crop disease detection and external APIs for fetching real-time weather information and supplementary agricultural data.

Data Flow

Users interact with the application through the web interface.

The system verifies user identity using secure authentication mechanisms.

The backend processes requests, interacts with the database and external APIs.

Processed results are returned to the frontend for display.

User notifications and updates are generated as required.

Chapter 7

ML Module

CHAPTER 7

ML MODULE

7.1 INTRODUCTION

The Machine Learning (ML) module is a vital component of the **AI-Powered Crop Disease Detection for Food Security**. Its primary objective is to provide intelligent, automated crop disease identification by analyzing plant leaf images. The ML module assists farmers and agricultural users by accurately detecting diseases at an early stage, enabling timely treatment and preventive action to reduce crop losses and improve yield.

7.2 PROBLEM DEFINITION AND ALGORITHM

Traditional crop disease detection methods rely on manual observation and expert knowledge, which can be time-consuming, inconsistent, and inaccessible to many farmers. These methods often lead to delayed diagnosis and improper treatment. To overcome these limitations, the ML module uses supervised learning techniques to automatically analyze crop images and identify disease patterns. By training the model on labeled plant disease datasets, the system can generate accurate predictions, thereby improving reliability, consistency, and efficiency in crop disease diagnosis. **Algorithm Used**

To achieve this, the system employs a **supervised learning approach**, specifically using the **Linear Regression algorithm**. This algorithm is chosen due to its simplicity, interpretability, and ability to capture linear relationships between multiple academic performance metrics and the final appraisal score. The model learns from historical appraisal data and identifies patterns that contribute to overall faculty performance, which are later used to predict outcomes for new faculty inputs.

7.3 ML MODEL DESCRIPTION

The ML module is developed using Python and machine learning libraries such as TensorFlow/Keras, NumPy, OpenCV, and Flask, implementing a complete workflow from data preprocessing to deployment.

Data Preparation & Exploration

The dataset contains labeled images of healthy and diseased crop leaves representing various plant diseases. Images are resized, normalized, and augmented to improve model generalization. The dataset is prepared using image processing techniques to enhance learning accuracy.

Model Selection & Training

A Convolutional Neural Network (CNN) model is trained to classify crop diseases. The dataset is divided into training and testing sets (80:20 ratio) to ensure unbiased evaluation. The model is trained over multiple epochs to optimize classification accuracy.

Model Evaluation

Model performance is evaluated using accuracy and loss metrics. Predictions made by the model are compared with actual labels to validate reliability and effectiveness in disease classification.

Model Serialization

After successful training, the trained model is saved in serialized format (e.g., model.h5) for future use. This allows the system to load the trained model without retraining, ensuring faster deployment.

Integration via Flask API

A lightweight Flask-based API acts as a bridge between the trained ML model and the web application. The API receives crop images through HTTP requests, preprocesses them, passes them to the trained model, and returns disease prediction results in JSON format.

7.4 RESULT ANALYSIS

The CNN-based model demonstrated reliable disease classification performance, as indicated by high accuracy on test data. The comparison between predicted and actual disease labels shows that the model effectively captures visual disease patterns from crop images. By deploying the trained model through a Flask API, the system enables real-time disease detection for users. The results validate the effectiveness of integrating machine learning into agricultural applications and indicate strong potential for future enhancements using larger datasets and advanced deep learning techniques.

Chapter 8

Coding

CHAPTER 8

CODING

8.1 CODE IMPLEMENTATION

The project uses **React (Vite)**, **Tailwind CSS**, and **JavaScript** for developing a responsive and user-friendly frontend interface. User authentication is implemented using **Firestore Authentication** to ensure secure access to the system. After successful login, users can upload crop or leaf images through structured input forms for disease analysis. The uploaded images are validated and sent to the backend server developed using **Flask (Python)**.

The backend processes the images and forwards them to the AI-based disease detection module for analysis. Detection results, including disease name, confidence score, symptoms, and treatment recommendations, are stored in **Firestore** under user-specific records. Weather-related data and detection history are also saved for future reference. All recent user activities such as image uploads and disease detections are logged and displayed dynamically on the user dashboard. The dashboard presents user-specific detection history, real-time weather updates, and analytical insights, ensuring an interactive and informative user experience.

Authentication

```
const Auth = () => {
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();
  const initialMode = searchParams.get('mode') || 'login';
  const [mode, setMode] = useState(initialMode);
  const { isDarkMode } = useTheme();

  const { login, signup, resetPassword, googleSignIn, currentUser, loading: authLoading } =
    useAuth();

  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState("");
  useEffect(() => {
    setMode(searchParams.get('mode') || 'login');
    setError("");
    setMessage("");
  }, [searchParams]);
  useEffect(() => {
    if (!authLoading && currentUser) {
      navigate('/dashboard');
    }
  })
}
```

```

    }, [currentUser, authLoading, navigate]));
const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setMessage("");
    setLoading(true);
    const formData = new FormData(e.target);
    const email = formData.get('email');
    const password = formData.get('password');
    try {
        if (mode === 'login') {
            await login(email, password);
            navigate('/dashboard');
        } else if (mode === 'signup') {
            await signup(email, password);
            navigate('/dashboard');
        } else if (mode === 'reset') {
            await resetPassword(email);
            setMessage('Check your inbox for further instructions');
        }
    } catch (err) {
        setError('Failed to ' + (mode === 'login' ? 'log in' : mode === 'signup' ? 'create account' : 'reset password') + ': ' + err.message);
    }
    setLoading(false);
};
const handleGoogleSignIn = async () => {
    try {
        setError("");
        setLoading(true);
        await googleSignIn();
        navigate('/dashboard');
    } catch (err) {
        setError('Failed to sign in with Google: ' + err.message);
    }
}

```

```

    }
    setLoading(false);
  };
  return (
    <div className={`min-h-screen flex flex-col justify-center py-12 sm:px-6 lg:px-8 transition-
colors duration-300 relative overflow-hidden`} >
      { /* Background Elements */ }
      <div className={`absolute inset-0 ${isDarkMode ? 'bg-gray-900' : 'bg-gradient-to-br from-
green-50 to-emerald-100'} -z-20`} />
      <div className="absolute top-0 left-0 w-full h-full overflow-hidden -z-10">
        <div className="absolute top-0 right-0 -mr-20 -mt-20 w-[600px] h-[600px] rounded-full
bg-primary/20 blur-3xl opacity-50" />
        <div className="absolute bottom-0 left-0 -ml-20 -mb-20 w-[600px] h-[600px] rounded-
full bg-secondary/20 blur-3xl opacity-50" />
      </div>
      <div className="sm:mx-auto sm:w-full sm:max-w-md relative z-10">
        <div className="flex justify-center mb-6">
          <Link to="/" className="flex items-center gap-2 group">
            <div className="p-2 rounded-xl bg-primary/10 group-hover:bg-primary/20
transition-colors duration-300">
              <Leaf className="h-10 w-10 text-primary" />
            </div>
            <span className="font-extrabold text-3xl bg-clip-text text-transparent bg-gradient-
to-r from-primary to-secondary tracking-tight">PlantGuard</span>
          </Link>
        </div>
        <h2 className={`mt-2 text-center text-3xl font-bold tracking-tight ${isDarkMode ? 'text-
white' : 'text-gray-900'}`} >
          {mode === 'login' && 'Welcome back'}
          {mode === 'signup' && 'Create your account'}
          {mode === 'reset' && 'Reset password'}
        </h2>
        <p className={`mt-2 text-center text-sm ${isDarkMode ? 'text-gray-400' : 'text-gray-
600'}`} >
          {mode === 'login' && (
            <div>
              New here?{' '}

```

```

        <button onClick={() => setMode('signup')} className="font-semibold text-
primary hover:text-emerald-600 transition-colors duration-200">

```

```

        Create an account

```

```

    </button>

```

```

  </>

```

```

)}

```

```

{mode === 'signup' && (

```

```

  <

```

```

    Already have an account?{' '}

```

```

    <button onClick={() => setMode('login')} className="font-semibold text-primary
hover:text-emerald-600 transition-colors duration-200">

```

```

    Sign in

```

```

  </button>

```

```

</>

```

```

)}

```

```

{mode === 'reset' && (

```

```

  <

```

```

    Remember it?{' '}

```

```

    <button onClick={() => setMode('login')} className="font-semibold text-primary
hover:text-emerald-600 transition-colors duration-200">

```

```

    Sign in

```

```

  </button>

```

```

</>

```

```

)}

```

```

</p>

```

```

</div>

```

```

<div className="mt-8 sm:mx-auto sm:w-full sm:max-w-md relative z-10">

```

```

  <div className={`py-8 px-4 shadow-xl sm:rounded-2xl sm:px-10 backdrop-blur-md
transition-all duration-300 ${isDarkMode ? 'bg-gray-800/80 border border-gray-700' : 'bg-white/80
border border-gray-100'} `}>

```

```

    {error && (

```

```

      <div className="mb-4 bg-red-50/90 border border-red-200 text-red-700 px-4 py-3
rounded-lg flex items-center gap-2" role="alert">

```

```

        <svg className="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24
24"><path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M12 8v4m0
4h.01M21 12a9 9 0 11-18 0 9 9 0 118 0z" /></svg>

```

```

    {error}

```

```

    </div>
  )}
  {message && (
    <div className="mb-4 bg-green-50/90 border border-green-200 text-green-700 px-4
py-3 rounded-lg flex items-center gap-2" role="alert">
      <svg className="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24
24"><path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M5 13l4 4L19 7"
/></svg>
      {message}
    </div>
  )}
  <form className="space-y-6" onSubmit={handleSubmit}>
    {mode === 'signup' && (
      <div>
        <label htmlFor="name" className={`block text-sm font-medium ${isDarkMode
? 'text-gray-300' : 'text-gray-700'} mb-1`} >
          Full Name
        </label>
        <div className="relative rounded-lg shadow-sm">
          <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-
events-none">
            <User className={`h-5 w-5 ${isDarkMode ? 'text-gray-500' : 'text-gray-
400'} `} />
          </div>
          <input
            id="name"
            name="name"
            type="text"
            autoComplete="name"
            required
            className={`block w-full pl-10 sm:text-sm rounded-lg transition-all
duration-200 focus:ring-2 focus:ring-primary/20 focus:border-primary py-2.5 ${isDarkMode ? 'bg-
gray-700/50 border-gray-600 text-white placeholder-gray-500' : 'bg-gray-50 border-gray-300 text-
gray-900 placeholder-gray-400'} `}
            placeholder="John Doe"
          />
        </div>
      )}
    )}
  </form>

```

```

    </div>
  })
  <div>
    <label htmlFor="email" className={`block text-sm font-medium ${isDarkMode ?
'text-gray-300' : 'text-gray-700'} mb-1`} >
      Email address
    </label>
    <div className="relative rounded-lg shadow-sm">
      <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-
none">
        <Mail className={`h-5 w-5 ${isDarkMode ? 'text-gray-500' : 'text-gray-
400'}}` />
      </div>
      <input
        id="email"
        name="email"
        type="email"
        autoComplete="email"
        required
        className={`block w-full pl-10 sm:text-sm rounded-lg transition-all duration-
200 focus:ring-2 focus:ring-primary/20 focus:border-primary py-2.5 ${isDarkMode ? 'bg-gray-
700/50 border-gray-600 text-white placeholder-gray-500' : 'bg-gray-50 border-gray-300 text-gray-
900 placeholder-gray-400'}}`
        placeholder="you@example.com"
      />
    </div>
  </div>
  {mode !== 'reset' && (
    <div>
      <label htmlFor="password" className={`block text-sm font-medium
${isDarkMode ? 'text-gray-300' : 'text-gray-700'} mb-1`} >
        Password
      </label>
      <div className="relative rounded-lg shadow-sm">
        <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-
events-none">

```

```

400'}}` } />

<Lock className={`h-5 w-5 ${isDarkMode ? 'text-gray-500' : 'text-gray-
400'}}` } />

</div>

<input
  id="password"
  name="password"
  type="password"
  autoComplete="current-password"
  required
  className={`block w-full pl-10 sm:text-sm rounded-lg transition-all
duration-200 focus:ring-2 focus:ring-primary/20 focus:border-primary py-2.5 ${isDarkMode ? 'bg-
gray-700/50 border-gray-600 text-white placeholder-gray-500' : 'bg-gray-50 border-gray-300 text-
gray-900 placeholder-gray-400'}}` }
  placeholder="....."
/>
</div>
</div>
)}
{mode === 'login' && (
  <div className="flex items-center justify-between">
    <div className="flex items-center">
      <input
        id="remember-me"
        name="remember-me"
        type="checkbox"
        className={`h-4 w-4 text-primary focus:ring-primary border rounded
${isDarkMode ? 'bg-gray-700 border-gray-600' : 'border-gray-300'}}` }
      />
      <label htmlFor="remember-me" className={`ml-2 block text-sm
${isDarkMode ? 'text-gray-300' : 'text-gray-600'}}` }>
        Remember me
      </label>
    </div>
    <div className="text-sm">
      <button
        type="button"

```

```

        onClick={() => setMode('reset')}

        className="font-medium text-primary hover:text-emerald-600 transition-
colors"

      >

        Forgot password?

      </button>

    </div>

  </div>

)}

<div>

  <button

    type="submit"

    disabled={loading}

    className="w-full flex justify-center py-3 px-4 border border-transparent
rounded-xl shadow-lg shadow-primary/20 text-sm font-bold text-white bg-gradient-to-r from-primary
to-secondary hover:from-emerald-600 hover:to-emerald-500 focus:outline-none focus:ring-2
focus:ring-offset-2 focus:ring-primary transition-all duration-300 transform hover:-translate-y-0.5
disabled:opacity-50 disabled:cursor-not-allowed"

  >

    {loading ? (

      <span className="flex items-center gap-2">

        <svg className="animate-spin h-5 w-5 text-white" viewBox="0 0 24 24">

          <circle className="opacity-25" cx="12" cy="12" r="10"
stroke="currentColor" strokeWidth="4"></circle>

          <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-
8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014 12H0c0 3.042 1.135 5.824 3 7.938l3-
2.647z"></path>

        </svg>

        Processing...

      </span>

    ) : (

      mode === 'login' ? 'Sign in' : mode === 'signup' ? 'Create Account' : 'Send
Reset Link'

    )}

  </button>

</div>

</form>

```



```

<div className="mt-8">
  <div className="relative">
    <div className="absolute inset-0 flex items-center">
      <div className={`w-full border-t ${isDarkMode ? 'border-gray-700' : 'border-
gray-200'}}` />
    </div>
    <div className="relative flex justify-center text-sm">
      <span className={`px-4 rounded-full ${isDarkMode ? 'bg-gray-800 text-gray-
400' : 'bg-white text-gray-500'}}`>Or continue with</span>
    </div>
  </div>
  <div className="mt-6 grid grid-cols-2 gap-4">
    <div>
      <button
        onClick={handleGoogleSignIn}
        disabled={loading}
        className={`w-full inline-flex justify-center items-center py-2.5 px-4 border
rounded-xl shadow-sm text-sm font-medium transition-all duration-200 ${isDarkMode ? 'border-
gray-600 bg-gray-700/50 text-gray-200 hover:bg-gray-600' : 'border-gray-200 bg-white text-gray-600
hover:bg-gray-50 hover:border-gray-300'}}`
      >
      <svg className="h-5 w-5 mr-2" viewBox="0 0 24 24">
        <path d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-
1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z" fill="#4285F4" />
        <path d="M12 23c2.97 0 5.46-.98 7.28-2.66l3.57-2.77c-.98.66-2.23 1.06-
3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z" fill="#34A853" />
        <path d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-
2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.84z" fill="#FBBC05" />
        <path d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1
12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z" fill="#EA4335" />
      </svg>
      Google
    </button>
  </div>
  <div>
    <button
      disabled={loading}

```

```

        className={`w-full inline-flex justify-center items-center py-2.5 px-4 border
rounded-xl shadow-sm text-sm font-medium transition-all duration-200 ${isDarkMode ? 'border-
gray-600 bg-gray-700/50 text-gray-200 hover:bg-gray-600' : 'border-gray-200 bg-white text-gray-600
hover:bg-gray-50 hover:border-gray-300'}`}

```

```

    >

```

```

        <svg className="h-5 w-5 mr-2 text-blue-600" fill="currentColor"
viewBox="0 0 20 20" aria-hidden="true">

```

```

        <path

```

```

            fillRule="evenodd"

```

```

            d="M20 10c0-5.523-4.477-10-10-10S0 4.477 0 10c0 4.991 3.657 9.128
8.438 9.878v-6.987h-2.54V10h2.54V7.797c0-2.506 1.492-3.89 3.777-3.89 1.094 0 2.238.195
2.238.195v2.46h-1.26c-1.243 0-1.63.771-1.63 1.562V10h2.773l-.443 2.89h-2.33v6.988C16.343
19.128 20 14.991 20 10z"

```

```

            clipRule="evenodd"

```

```

        />

```

```

    </svg>

```

```

    Facebook

```

```

</button>

```

```

</div>

```

```

</div>

```

```

</div>

```

```

</div>

```

```

</div>

```

```

</div>

```

```

);

```

```

};

```

```

export default Auth;

```

Landing Page

```

const LandingPage = () => {
  const { isDarkMode } = useTheme();
  const { currentUser } = useAuth();
  const isAuthenticated = !!currentUser;
  const navigate = useNavigate();
  useEffect(() => {
    if (isAuthenticated) {
      navigate('/dashboard');
    }
  })
}

```

```

}, [isAuthenticated, navigate]);

const features = [
  {
    icon: ShieldCheck,
    title: 'Instant Disease Detection',
    description: 'Upload a photo of your crop leaf and get instant results with high accuracy using our advanced AI models trained on thousands of crop diseases.',
    color: 'bg-green-100 text-green-600 dark:bg-green-900 dark:text-green-400'
  },
  {
    icon: Zap,
    title: 'Treatment Recommendations',
    description: 'Get actionable advice and treatment steps for detected diseases to save your crops immediately. Our recommendations are based on agricultural best practices.',
    color: 'bg-yellow-100 text-yellow-600 dark:bg-yellow-900 dark:text-yellow-400'
  },
  {
    icon: BarChart3,
    title: 'Detection History & Analytics',
    description: 'Keep track of all your past scans and monitor the health trends of your farm over time with detailed charts and insights.',
    color: 'bg-blue-100 text-blue-600 dark:bg-blue-900 dark:text-blue-400'
  },
  {
    icon: Smartphone,
    title: 'Government Schemes',
    description: 'Access the latest government schemes and subsidies available for farmers in your region. Stay updated with new programs and benefits.',
    color: 'bg-purple-100 text-purple-600 dark:bg-purple-900 dark:text-purple-400'
  },
  {
    icon: Camera,
    title: 'Easy Image Upload',
    description: 'Simple and intuitive interface to capture or upload crop images. Works seamlessly on mobile devices for on-field detection.',
    color: 'bg-red-100 text-red-600 dark:bg-red-900 dark:text-red-400'
  }
]

```

```

    }
  ];
  return (
    <div className={`min-h-screen ${isDarkMode ? 'bg-gray-950' : 'bg-gray-50'} transition-colors duration-300`} >
      <div className="relative overflow-hidden">
        <div className={`absolute inset-0 ${isDarkMode ? 'bg-gray-900' : 'bg-gradient-to-br from-green-50 to-emerald-100'} -z-20`} />
        { /* Abstract shapes */ }
        <div className="absolute top-0 right-0 -mr-20 -mt-20 w-[600px] h-[600px] rounded-full bg-primary/20 blur-3xl -z-10" />
        <div className="absolute bottom-0 left-0 -ml-20 -mb-20 w-[600px] h-[600px] rounded-full bg-secondary/20 blur-3xl -z-10" />
        <div className="max-w-7xl mx-auto">
          <div className="relative z-10 pb-8 sm:pb-16 md:pb-20 lg:w-1/2 lg:max-w-none lg:pb-28 xl:pb-32 pt-24 px-4 sm:px-6 lg:px-8 lg:pr-20">
            <main className="mt-10 mx-auto max-w-7xl sm:mt-12 md:mt-16 lg:mt-20 xl:mt-28">
              <div className="sm:text-center lg:text-left">
                <div className="inline-flex items-center px-4 py-2 rounded-full bg-primary/10 text-primary mb-6 animate-fade-in-up">
                  <span className="text-sm font-semibold tracking-wide uppercase">AI-Powered Agriculture</span>
                </div>
                <h1 className={`text-4xl tracking-tight font-extrabold sm:text-5xl md:text-6xl ${isDarkMode ? 'text-white' : 'text-gray-900'} mb-6`} >
                  <span className="block xl:inline">Protect your crops with</span>{' '}
                  <span className="block text-transparent bg-clip-text bg-gradient-to-r from-primary to-secondary xl:inline">AI Precision</span>
                </h1>
                <p className={`mt-3 text-base sm:mt-5 sm:text-lg md:mt-5 md:text-xl ${isDarkMode ? 'text-gray-300' : 'text-gray-600'} leading-relaxed`} >
                  Detect diseases early, get treatment recommendations, and maximize your yield with our smart crop disease detection system. Built for modern farmers.
                </p>
                <div className="mt-8 sm:mt-10 sm:flex sm:justify-center lg:justify-start gap-4">
                  {!isAuthenticated ? (
                    <Link

```

```

        to="/auth"

        className="w-full sm:w-auto flex items-center justify-center px-8 py-3
border border-transparent text-base font-medium rounded-xl text-white bg-primary hover:bg-
emerald-600 md:py-4 md:text-lg md:px-10 transition-all duration-300 shadow-lg hover:shadow-
primary/30 transform hover:-translate-y-1"

    >

        Get Started

    </Link>

) : (

    <Link

        to="/dashboard"

        className="w-full sm:w-auto flex items-center justify-center px-8 py-3
border border-transparent text-base font-medium rounded-xl text-white bg-primary hover:bg-
emerald-600 md:py-4 md:text-lg md:px-10 transition-all duration-300 shadow-lg hover:shadow-
primary/30 transform hover:-translate-y-1"

    >

        Go to Dashboard

    </Link>

)}

<button

    onClick={() => document.getElementById('features').scrollIntoView({
behavior: 'smooth' })}

    className={`mt-3 sm:mt-0 w-full sm:w-auto flex items-center justify-
center px-8 py-3 border border-transparent text-base font-medium rounded-xl md:py-4 md:text-lg
md:px-10 transition-all duration-300 ${isDarkMode ? 'bg-gray-800 text-gray-200 hover:bg-gray-700'
: 'bg-white text-gray-700 hover:bg-gray-50'} shadow-md hover:shadow-lg`}

    >

        Learn More

    </button>

</div>

</div>

</main>

</div>

</div>

<div className="lg:absolute lg:inset-y-0 lg:right-0 lg:w-1/2 overflow-hidden">

    <div className="h-56 w-full sm:h-72 md:h-96 lg:w-full lg:h-full relative">

        <div className="absolute inset-0 bg-gradient-to-r from-transparent to-black/20 z-
10"></div>

```

```

        
    </div>
</div>
</div>
{ /* Features Section */}

<div id="features" className={`py-12 ${isDarkMode ? 'bg-gray-900' : 'bg-white'} transition-
colors duration-300`} >
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="mt-10">
            <div className="grid grid-cols-1 gap-8 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4">
                {features.map((feature, index) => (
                    <div key={index} className={`rounded-lg shadow-md hover:shadow-xl
transition-all duration-300 p-6 transform hover:-translate-y-1 ${isDarkMode ? 'bg-gray-800
hover:bg-gray-700' : 'bg-white'}`} >
                        <div className={`inline-flex items-center justify-center h-12 w-12 rounded-
md ${feature.color} mb-4`} >
                            <feature.icon className="h-6 w-6" aria-hidden="true" />
                        </div>
                        <h3 className={`text-lg font-semibold mb-2 ${isDarkMode ? 'text-white' :
'text-gray-900'}`} >
                            {feature.title}
                        </h3>
                        <p className={`text-sm leading-relaxed ${isDarkMode ? 'text-gray-400' :
'text-gray-600'}`} >
                            {feature.description}
                        </p>
                    </div>
                )))}
            </div>
        </div>
    </div>

```

```

</div>

</div>

{ /* About Section */ }

<div id="about" className={ `py-12 ${isDarkMode ? 'bg-gray-950' : 'bg-gray-50'} transition-
colors duration-300` }>

  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">

    <div className="lg:grid lg:grid-cols-2 lg:gap-8 lg:items-center mb-16">

      <div>

        <h3 className={ `text-2xl font-extrabold sm:text-3xl ${isDarkMode ? 'text-white' :
'text-gray-900'} ` }>
          Empowering Farmers
        </h3>

        <p className={ `mt-4 text-lg ${isDarkMode ? 'text-gray-300' : 'text-gray-600'} ` }>
          At PlantGuard, we're committed to revolutionizing agriculture through
          technology. Our mission is to provide farmers with accessible, accurate, and actionable insights to
          protect their crops from diseases.
        </p>

        <p className={ `mt-4 text-lg ${isDarkMode ? 'text-gray-300' : 'text-gray-600'} ` }>
          By leveraging advanced artificial intelligence and machine learning, we help
          farmers detect crop diseases early, receive treatment recommendations, and make informed decisions
          that lead to healthier crops and better yields.
        </p>
      </div>
    </div>

    <div className="mt-10 lg:mt-0">

      
    </div>
  </div>
</div>

{ /* Values Grid */ }

<div className="grid grid-cols-1 gap-8 sm:grid-cols-2 lg:grid-cols-4">

  <div className="text-center">

    <div className="flex items-center justify-center h-16 w-16 rounded-full bg-green-
100 text-green-600 dark:bg-green-900 dark:text-green-400 mx-auto mb-4">

```

```

        <Users className="h-8 w-8" />
    </div>

    <h3 className={`text-lg font-semibold mb-2 ${isDarkMode ? 'text-white' : 'text-gray-900'}}`>Farmer-Centric</h3>

    <p className={isDarkMode ? 'text-gray-400' : 'text-gray-600'}>
        We put farmers first in everything we design and build.
    </p>
</div>

<div className="text-center">
    <div className="flex items-center justify-center h-16 w-16 rounded-full bg-blue-100 text-blue-600 dark:bg-blue-900 dark:text-blue-400 mx-auto mb-4">
        <Target className="h-8 w-8" />
    </div>

    <h3 className={`text-lg font-semibold mb-2 ${isDarkMode ? 'text-white' : 'text-gray-900'}}`>Accuracy</h3>

    <p className={isDarkMode ? 'text-gray-400' : 'text-gray-600'}>
        We strive for the highest accuracy in disease detection.
    </p>
</div>

<div className="text-center">
    <div className="flex items-center justify-center h-16 w-16 rounded-full bg-yellow-100 text-yellow-600 dark:bg-yellow-900 dark:text-yellow-400 mx-auto mb-4">
        <Award className="h-8 w-8" />
    </div>

    <h3 className={`text-lg font-semibold mb-2 ${isDarkMode ? 'text-white' : 'text-gray-900'}}`>Innovation</h3>

    <p className={isDarkMode ? 'text-gray-400' : 'text-gray-600'}>
        We continuously innovate to serve farmers better.
    </p>
</div>

<div className="text-center">
    <div className="flex items-center justify-center h-16 w-16 rounded-full bg-red-100 text-red-600 dark:bg-red-900 dark:text-red-400 mx-auto mb-4">
        <Heart className="h-8 w-8" />
    </div>

    <h3 className={`text-lg font-semibold mb-2 ${isDarkMode ? 'text-white' : 'text-gray-900'}}`>Accessibility</h3>

```



```

        <p className={isDarkMode ? 'text-gray-400' : 'text-gray-600'}>
            We make advanced technology accessible to all farmers.
        </p>
    </div>
</div>
</div>
</div>
    { /* Footer */ }
    <footer className={` ${isDarkMode ? 'bg-gray-800' : 'bg-gray-800'} transition-colors
duration-300`} >
        <div className="max-w-7xl mx-auto py-12 px-4 sm:px-6 lg:px-8">
            <p className="text-center text-base text-gray-400">
                &copy; 2025 PlantGuard. All rights reserved.
            </p>
        </div>
    </footer>
</div>
);
};

```

```
export default LandingPage;
```

Dashboard

```

const Dashboard = () => {
    const { isDarkMode } = useTheme();
    const { currentUser } = useAuth();
    const [history, setHistory] = useState([]);
    const [loading, setLoading] = useState(true);
    const [weather, setWeather] = useState(null);
    const [weatherLoading, setWeatherLoading] = useState(true);
    const [locationError, setLocationError] = useState("");
    useEffect(() => {
        const fetchHistory = async () => {
            if (currentUser) {
                try {

```

```

        const response = await getDetectionHistory();
        setHistory(Array.isArray(response.data) ? response.data : []);
    } catch (error) {
        console.error("Failed to fetch history:", error);
    } finally {
        setLoading(false);
    }
} else {
    setLoading(false);
}
};

fetchHistory();
}, [currentUser]);

const fetchWeather = async (forceRefresh = false) => {
    setWeatherLoading(true);
    setLocationError("");
    if (forceRefresh) {
        setWeather(null);
    }
    if (!forceRefresh) {
        const cachedWeather = localStorage.getItem('weatherData');
        const cachedTimestamp = localStorage.getItem('weatherTimestamp');
        if (cachedWeather && cachedTimestamp) {
            const now = Date.now();
            const oneHour = 60 * 60 * 1000;
            if (now - parseInt(cachedTimestamp) < oneHour) {
                setWeather({ ...JSON.parse(cachedWeather) });
                setWeatherLoading(false);
                return;
            }
        }
    }
    if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(

```

```

    async (position) => {
      try {
        const { latitude, longitude } = position.coords;
        const response = await getWeather(latitude, longitude);
        const weatherData = response.data;
        localStorage.setItem('weatherData', JSON.stringify(weatherData));
        localStorage.setItem('weatherTimestamp', Date.now().toString());
        setWeather({ ...weatherData });
      } catch (error) {
        console.error("Failed to fetch weather:", error);
        setLocationError('Failed to fetch weather data');
      } finally {
        setWeatherLoading(false);
      }
    },
    (error) => {
      console.error("Geolocation error:", error);
      setLocationError('Location access denied. Please enable location to see weather.');
```

setWeatherLoading(false);

```

    }
  );
} else {
  setLocationError('Geolocation is not supported by your browser');
  setWeatherLoading(false);
}
};

useEffect(() => {
  fetchWeather();
}, []);

return (
  <div className={`min-h-screen ${isDarkMode ? 'bg-gray-950' : 'bg-gray-50'} transition-colors duration-300`}>
    <main className="max-w-7xl mx-auto py-8 px-4 sm:px-6 lg:px-8 pt-24">
      { /* Page Header */ }
    </main>
  </div>
);

```

```

<div className="mb-8">

  <h1 className={`text-3xl font-bold ${isDarkMode ? 'text-white' : 'text-gray-900'}}`}>Dashboard</h1>

  <p className={`mt-2 text-sm ${isDarkMode ? 'text-gray-400' : 'text-gray-600'}}`}>

    Welcome back{currentUser?.displayName ? `, ${currentUser.displayName}` : currentUser?.email ? `, ${currentUser.email}` : ""}! Here's an overview of your farm's status.

  </p>
</div>

{ /* Weather Section */ }

{weatherLoading ? (

  <div className={`bg-gradient-to-r ${isDarkMode ? 'from-blue-900 to-blue-800' : 'from-blue-500 to-blue-600'}`} rounded-2xl shadow-lg overflow-hidden mb-8 text-white p-10`}>

    <div className="flex items-center justify-center">

      <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-white"></div>

      <span className="ml-3">Loading weather...</span>

    </div>

  </div>

) : locationError ? (

  <div className={`rounded-2xl shadow-lg overflow-hidden mb-8 p-6 ${isDarkMode ? 'bg-yellow-900/20 border border-yellow-800' : 'bg-yellow-50 border border-yellow-200'}}`}>

    <div className="flex items-center">

      <AlertTriangle className={`h-6 w-6 ${isDarkMode ? 'text-yellow-400' : 'text-yellow-600'}}` />

      <p className={`ml-3 text-sm ${isDarkMode ? 'text-yellow-300' : 'text-yellow-800'}}`>{locationError}</p>

    </div>

  </div>

) : weather && (

  <div className={`bg-gradient-to-r ${isDarkMode ? 'from-blue-900 to-blue-800' : 'from-blue-500 to-blue-600'}`} rounded-2xl shadow-lg overflow-hidden mb-8 text-white transition-all duration-300`}>

    <div className="p-6 sm:p-10">

      <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center">

        <div>

          <div className="flex items-center gap-3">

```

```

        <h2 className={`text-lg font-medium ${isDarkMode ? 'text-blue-200' :
'text-blue-100'}}`>Current Weather</h2>

        <button
            onClick={() => fetchWeather(true)}
            className={`p-2 rounded-lg transition-all duration-200 hover:scale-110
${isDarkMode ? 'bg-blue-700 hover:bg-blue-600' : 'bg-blue-400/30 hover:bg-blue-400/50'}}`
            title="Refresh weather"
        >

            <RefreshCw className="h-4 w-4 text-white" />

        </button>

    </div>

    <div className="mt-2 flex items-baseline">

        <span className="text-5xl font-extrabold tracking-tight">{weather.temperature}°C</span>

        <span className={`ml-2 text-xl font-medium ${isDarkMode ? 'text-blue-200' : 'text-blue-100'}}`>{weather.description}</span>

    </div>

    <p className={`mt-1 text-sm flex items-center ${isDarkMode ? 'text-blue-200' : 'text-blue-100'}}`>

        <MapPin className="h-4 w-4 mr-1" />

        {weather.location}

    </p>

</div>

<div className="mt-6 sm:mt-0">

    <CloudSun className={`h-20 w-20 ${isDarkMode ? 'text-blue-300' : 'text-blue-200'} opacity-80`} />

</div>

</div>

    {/* Weather Details Grid */}

    <div className={`mt-8 grid grid-cols-1 gap-4 sm:grid-cols-3 pt-6 border-t
${isDarkMode ? 'border-blue-700' : 'border-blue-400'}}`>

        <div className="flex items-center">

            <div className={`p-2 rounded-lg ${isDarkMode ? 'bg-blue-700' : 'bg-blue-400 bg-opacity-30'}}`>

                <Wind className="h-6 w-6 text-white" />

            </div>

            <div className="ml-4">

```

```

        <p className={`text-sm font-medium ${isDarkMode ? 'text-blue-200' :
'text-blue-100'}}`>Wind Speed</p>

        <p className="text-lg font-bold">{weather.wind_speed} km/h</p>
    </div>
</div>

<div className="flex items-center">

    <div className={`p-2 rounded-lg ${isDarkMode ? 'bg-blue-700' : 'bg-blue-
400 bg-opacity-30'}}`>

        <Droplets className="h-6 w-6 text-white" />

    </div>

    <div className="ml-4">

        <p className={`text-sm font-medium ${isDarkMode ? 'text-blue-200' :
'text-blue-100'}}`>Humidity</p>

        <p className="text-lg font-bold">{weather.humidity}%</p>

    </div>
</div>

<div className="flex items-center">

    <div className={`p-2 rounded-lg ${isDarkMode ? 'bg-blue-700' : 'bg-blue-
400 bg-opacity-30'}}`>

        <Thermometer className="h-6 w-6 text-white" />

    </div>

    <div className="ml-4">

        <p className={`text-sm font-medium ${isDarkMode ? 'text-blue-200' :
'text-blue-100'}}`>Feels Like</p>

        <p className="text-lg font-bold">{weather.feels_like}°C</p>

    </div>
</div>
</div>
</div>
)}

{/* Quick Actions Grid */}

<h2 className={`text-xl font-bold mb-6 ${isDarkMode ? 'text-white' : 'text-gray-
900'}}`>Quick Actions</h2>

<div className="grid grid-cols-1 gap-6 sm:grid-cols-2 lg:grid-cols-3 mb-8">

    {/* Disease Detection Card */}

```

```

<Link to="/detect" className={`group relative rounded-xl shadow-sm hover:shadow-
md transition-all duration-200 p-6 border ${isDarkMode ? 'bg-gray-900 border-gray-700
hover:border-green-600' : 'bg-white border-gray-100 hover:border-green-200'}}`>

  <div className="flex items-center justify-between">

    <div>

      <h3 className={`text-lg font-semibold mb-2 ${isDarkMode ? 'text-white' : 'text-
gray-900'}}`>Disease Detection</h3>

      <p className={`text-sm ${isDarkMode ? 'text-gray-400' : 'text-gray-
600'}}`>Scan your crops for diseases</p>

    </div>

    <div className={`p-3 rounded-lg ${isDarkMode ? 'bg-green-900 text-green-400' :
'bg-green-100 text-green-600'}}`>

      <Scan className="h-6 w-6" />

    </div>

  </div>

</Link>

{ /* Reports */ }

<Link to="/reports" className={`group relative rounded-xl shadow-sm hover:shadow-
md transition-all duration-200 p-6 border ${isDarkMode ? 'bg-gray-900 border-gray-700
hover:border-blue-600' : 'bg-white border-gray-100 hover:border-blue-200'}}`>

  <div className="flex items-center justify-between">

    <div>

      <h3 className={`text-lg font-semibold mb-2 ${isDarkMode ? 'text-white' : 'text-
gray-900'}}`>Detection History</h3>

      <p className={`text-sm ${isDarkMode ? 'text-gray-400' : 'text-gray-
600'}}`>View past scans and reports</p>

    </div>

    <div className={`p-3 rounded-lg ${isDarkMode ? 'bg-blue-900 text-blue-400' :
'bg-blue-100 text-blue-600'}}`>

      <FileText className="h-6 w-6" />

    </div>

  </div>

</Link>

{ /* Schemes Card */ }

<Link to="/schemes" className={`group relative rounded-xl shadow-sm hover:shadow-
md transition-all duration-200 p-6 border ${isDarkMode ? 'bg-gray-900 border-gray-700
hover:border-purple-600' : 'bg-white border-gray-100 hover:border-purple-200'}}`>

  <div className="flex items-center justify-between">

    <div>

```

```

        <h3 className={`text-lg font-semibold mb-2 ${isDarkMode ? 'text-white' : 'text-gray-900'}}`>Government Schemes</h3>

        <p className={`text-sm ${isDarkMode ? 'text-gray-400' : 'text-gray-600'}}`>Explore available schemes</p>

    </div>

    <div className={`p-3 rounded-lg ${isDarkMode ? 'bg-purple-900 text-purple-400' : 'bg-purple-100 text-purple-600'}}`>

        <Landmark className="h-6 w-6" />

    </div>

</div>

</Link>

</div>

{ /* Recent Detections */ }

<div className="mb-8">

    <div className="flex justify-between items-center mb-6">

        <h2 className={`text-xl font-bold ${isDarkMode ? 'text-white' : 'text-gray-900'}}`>Recent Detections</h2>

        <Link to="/reports" className={`text-sm font-medium ${isDarkMode ? 'text-green-400 hover:text-green-300' : 'text-green-600 hover:text-green-700'}}`>

            View All →

        </Link>

    </div>

    {loading ? (

        <div className="flex items-center justify-center py-12">

            <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-green-600"></div>

        </div>

    ) : history.length === 0 ? (

        <div className={`text-center py-12 rounded-xl border border-dashed ${isDarkMode ? 'border-gray-700 bg-gray-900/50' : 'border-gray-300 bg-white/50'}}`>

            <Scan className={`mx-auto h-12 w-12 ${isDarkMode ? 'text-gray-600' : 'text-gray-400'}}` />

            <h3 className={`mt-2 text-sm font-medium ${isDarkMode ? 'text-white' : 'text-gray-900'}}`>No detections yet</h3>

            <p className={`mt-1 text-sm ${isDarkMode ? 'text-gray-400' : 'text-gray-500'}}`>Start by analyzing your first crop image.</p>

            <Link to="/detect" className="mt-4 inline-flex items-center px-4 py-2 bg-green-600 text-white rounded-lg hover:bg-green-700 transition-colors">

```



```

        Start Detection
      </Link>
    </div>
  ): (
    <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
      {history.slice(0, 3).map((item) => (
        <div key={item.id} className={`rounded-lg p-4 border ${isDarkMode ? 'bg-gray-900 border-gray-700' : 'bg-white border-gray-200'}`} >
          <div className="flex items-start justify-between">
            <div className="flex-1">
              <h4 className={`text-sm font-semibold ${isDarkMode ? 'text-white' : 'text-gray-900'} `}>{item.plant_name}</h4>
              <p className={`text-xs mt-1 ${isDarkMode ? 'text-gray-400' : 'text-gray-600'} `}>
                Confidence: {item.confidence}
              </p>
              <p className={`text-xs mt-1 ${isDarkMode ? 'text-gray-500' : 'text-gray-500'} `}>
                {new Date(item.detected_at).toLocaleDateString()}
              </p>
            </div>
            <CheckCircle className="h-5 w-5 text-green-600" />
          </div>
        </div>
      ))}
    </div>
  )}
</div>
</main>
</div>
);
};
export default Dashboard;

```

Chapter 9

Results And

Output Screens

CHAPTER 9

RESULT AND OUTPUT SCREENS

9.1 RESULTS

The implementation of the "**Higher Education Career Advancement & Monitoring System**" demonstrates a functional and efficient digital platform for managing faculty self-appraisals. The frontend, developed using Next JS, Tailwind CSS, and TypeScript, provides a clean and responsive interface for seamless user interaction. JWT Authentication enables secure and role based login functionality for faculty members and administrators.

9.2 OUTPUT SCREENS

Below are the output screens captured during the testing phase.

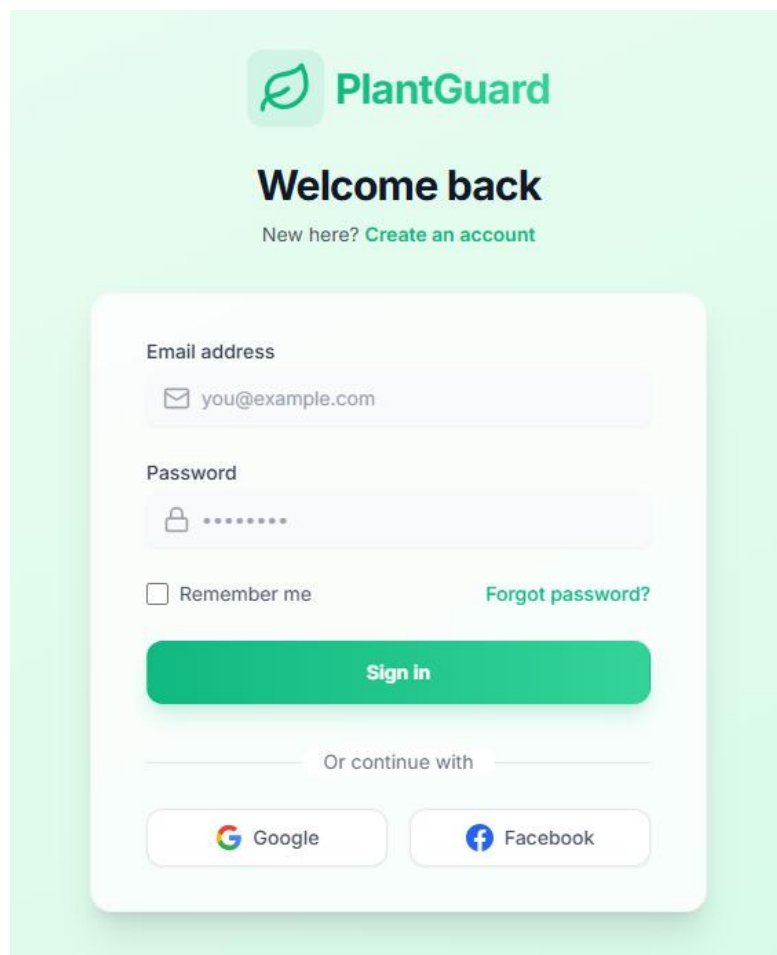
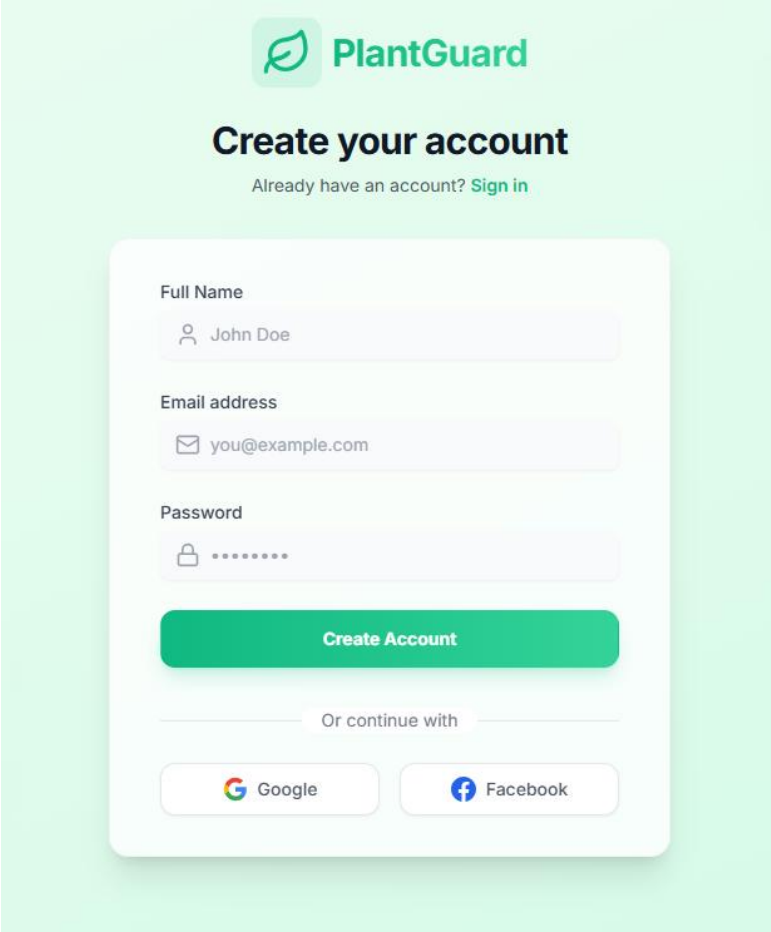


Figure 9.2.1: Login Page



The image shows a web form for creating a PlantGuard account. At the top, the PlantGuard logo is displayed. Below it, the heading "Create your account" is centered, followed by a link "Already have an account? Sign in". The form itself is a white card with three input fields: "Full Name" (with a person icon and the text "John Doe"), "Email address" (with an envelope icon and the text "you@example.com"), and "Password" (with a lock icon and masked dots). A green "Create Account" button is positioned below the password field. Underneath the button is a horizontal line with the text "Or continue with". At the bottom of the card are two buttons for social login: "Google" (with the Google logo) and "Facebook" (with the Facebook logo).

Figure 9.2.2: Signup Page

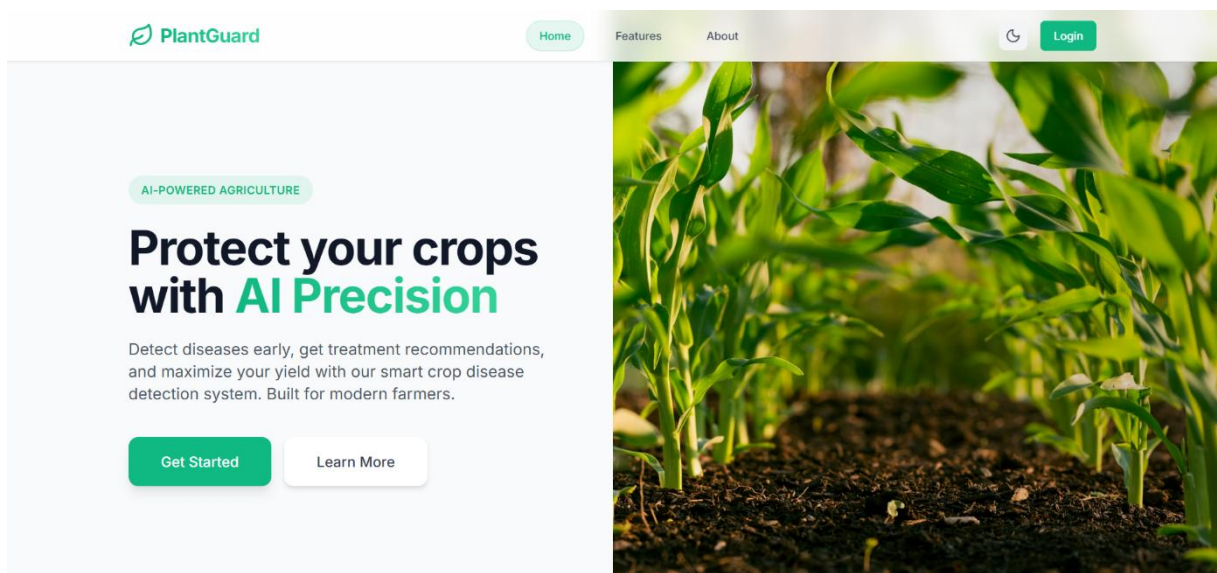


Figure 9.2.3: Home Page

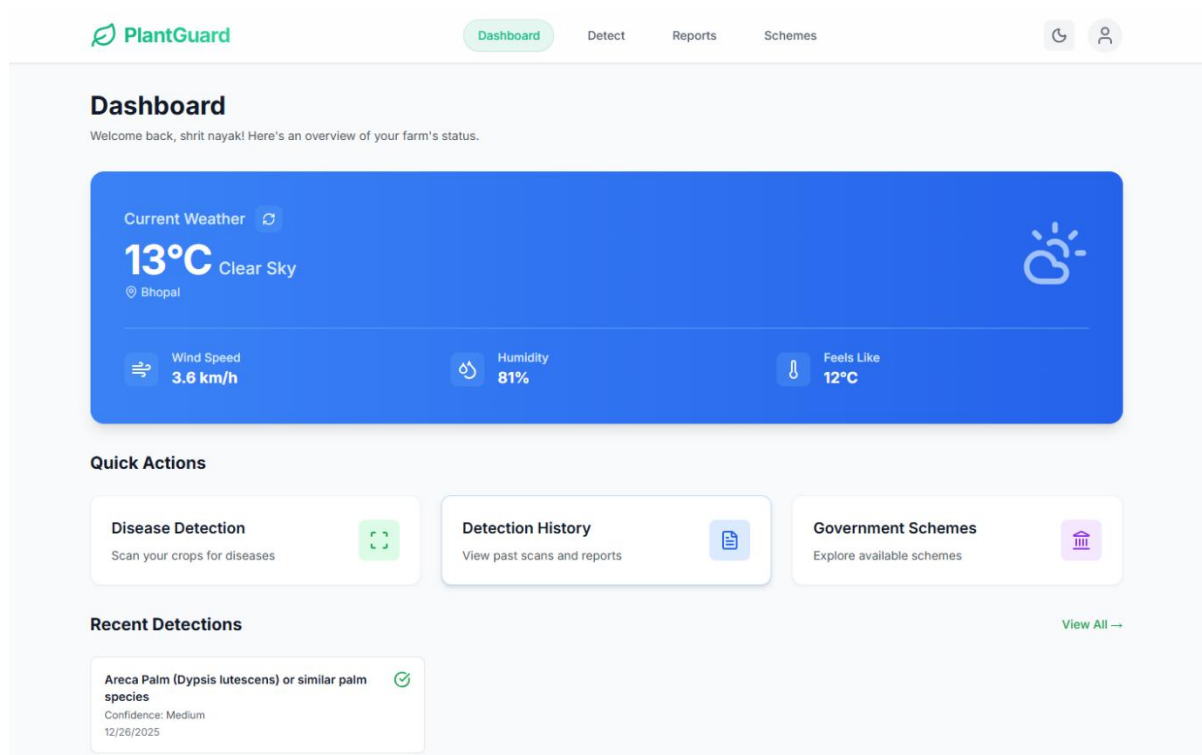


Figure 9.2.4: Dashboard Page

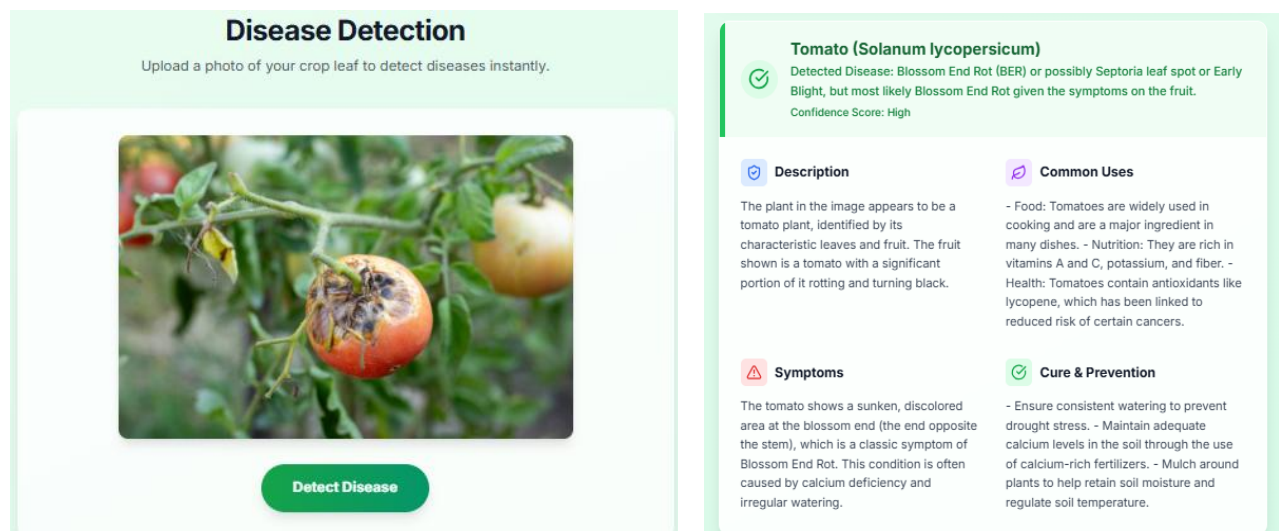


Figure 9.2.5: Disease Detection Page

Detection History

View and manage your past crop disease scans.

Tomato (<i>Solanum lycopersicum</i>) Confidence: High	The plant in the image appears to be a tomato plant, identified by its characteristic leaves and fruit. The fruit shown is a tomato with a... 📅 12/27/2025	 
Areca Palm (<i>Dypsis lutescens</i>) or similar palm species Confidence: Medium	The image shows a green leaf with multiple brown spots and lesions, some of which have a concentric ring pattern. The leaf appears to be from a... 📅 12/26/2025	 

Figure 9.2.6: Detection History Page

Government Schemes

Explore financial aid and subsidies available for you.

🔼 All Crops

Pradhan Mantri Fasal Bima Yojana (PMFBY) National

Crop insurance scheme to provide financial support to farmers suffering crop loss/damage arising out of unforeseen events.

Eligibility: All farmers growing notified crops in notified areas.

[Learn More](#)

Soil Health Card Scheme National

Government provides soil health cards to farmers which will carry crop-wise recommendations of nutrients and fertilizers.

Eligibility: All farmers.

[Learn More](#)

National Horticulture Mission National

Promotes holistic growth of the horticulture sector through area based regionally differentiated strategies.

Eligibility: Farmers with land ownership.

[Learn More](#)

Paramparagat Krishi Vikas Yojana National

Promotes organic farming through adoption of organic village by cluster approach and PGS certification.

Eligibility: Cluster of farmers.

[Learn More](#)

Figure 9.2.7: Govt. Schemes Page

Chapter 10

Conclusion and Future Work

CHAPTER 10

CONCLUSIONS AND FUTURE WORK

10.1 CONCLUSION

The developed web-based application streamlines the process of recording, verifying, and displaying faculty achievements such as publications, projects, and seminars. By integrating JWT for authentication and database management, the system ensures scalability, reliability, and ease of use. The DOI verification feature further enhances credibility by validating publication data against trusted external sources like CrossRef.

With role-based access control and real-time updates, the system empowers both faculty members and administrators to efficiently manage and track academic contributions. Overall, this solution addresses key challenges in academic data management and provides a robust platform for maintaining transparency and academic integrity in higher education institutions.

10.2 FUTURE WORK

While the current version of the system is functional, there are several potential improvements and extensions that can enhance its capabilities.

Advanced Search & Filters

Implementing keyword-based search, date range filters, and category sorting for better publication and project management.

Analytics Dashboard

Integrating data visualization to show trends in research output, collaboration networks, and indexing status.

Email Notifications

Alerting users about successful submissions, pending actions, and collaboration requests.

Integration with ORCID

Allowing users to import/export their academic data easily.

REFERENCES

JOURNALS / RESEARCH PAPERS

- [1]. https://ijirt.org/publishedpaper/IJIRT177145_PAPER.pdf
- [2]. <https://www.sciencedirect.com/science/article/pii/S2772375523000680>

WEBSITES

- [3]. <https://en.wikipedia.org/wiki/Plantix>
- [4]. <https://www.mdpi.com/2073-4395/12/10/2395>

PROJECT SUMMARY

About Project

Title of the project	AI-Powered Crop Disease Detection for Food Security
Semester	5 th
Members	04
Team Leader	Shivam Kumar
Describe role of every member in the project	Shivam Kumar : Backend Development, Authentication. Shrit Kumar : Frontend Development, APIs Integration Vivek Kumar : Frontend Designing & Development Yash Raj : Backend Development, Machine learning.
What is the motivation for selecting this project?	The motivation behind this project is to replace traditional and manual crop disease identification methods with a secure, AI-based digital platform that ensures accurate detection, real-time diagnosis, and data-driven agricultural decision-making, supporting improved crop health and sustainable farming practices..
Project Type (Desktop Application, Web Application, Mobile App, Web)	Web- Based Application

Tools & Technologies

Programming language used	React.JS, TailwindCSS, JavaScript,Python(flask)
IDE used (with version)	VSCode (1.106))
Front End Technologies (with version, wherever Applicable)	React (Vite), Tailwind CSS v3.x, JavaScript (ES6+), Axios v1.x
Back End Technologies (with version, wherever applicable)	Python 3.8+, Flask 2.x, Flask-SocketIO 5.x, Firebase Admin SDK
Database used (with version)	Firebase Firestore

Software Design & Coding

Is prototype of the software developed?	Yes
SDLC model followed (Waterfall, Agile, Spiral etc.)	Incremental
Why above SDLC model is followed?	The Incremental SDLC model is best suited for this project because the system is developed in small, functional modules that are built, tested, and refined based on user feedback, with features such as image upload, disease detection, and weather integration added incrementally until the final application is completed.
Justify that the SDLC model mentioned above is followed in the project.	Since the project requirements such as disease detection features, API integrations, and user interface enhancements evolved during development, the Incremental SDLC model was followed to incorporate changes and improvements efficiently at each stage.
Software Design approach followed (Functional or Object Oriented)	Object Oriented Approach
Name the diagrams developed (According to the Design approach followed)	Use Case Diagrams
In case Object Oriented approach is followed, which of the OOPS principles are covered in design?	Abstraction Polymorphism Encapsulation Inheritance
No. of Tiers (example 3-tier)	3
Total no. of front-end pages	5
Total no. of tables in database	6
Database in which Normal Form?	NH2
Are the entries in database encrypted?	Yes
Front end validations applied (Yes / No)	Yes
Session management done (in case of web applications)	Yes
Is application browser compatible (in case of web applications)	Yes
Exception handling done (Yes / No)	Yes

Commenting done in code (Yes / No)	Yes
Naming convention followed (Yes / No)	Yes
What difficulties faced during deployment of project?	Integration Challenges, API Integration, Version Control Issues, Dependency Management Complexity
Total no. of Use-cases	6
Give titles of Use-cases	User Registration and Login, Upload Crop Image, Crop Disease Detection, View Disease Diagnosis Result, View Weather Information, Manage Detection History

Project Requirements

MVC architecture followed (Yes / No)	Yes
If yes, write the name of MVC architecture followed (MVC-1, MVC-2)	MVC-2
Design Pattern used (Yes / No)	Yes
If yes, write the name of Design Pattern used	MVC
Interface type (CLI / GUI)	GUI
No. of Actors	2
Name of Actors	Farmer / Agricultural User, Admin
Total no. of Functional Requirements	6
List few important non-Functional Requirements	Performance, Reliability, Usability, Security

Testing

Which testing is performed? (Manual or Automation)	Manual
Is Beta testing done for this project?	No

Write project narrative covering above mentioned points

The AI-Powered Crop Disease Detection for Food Security was conceptualized and developed to address the challenges of traditional crop disease identification methods used in agriculture. These manual and expert-dependent practices were often time-consuming, inaccurate, and difficult to access, especially for farmers in rural areas. This project presents a modern, AI-based web solution that automates crop disease detection through image analysis, centralized data management, and real-time diagnosis. The platform is built using React, Tailwind CSS, and JavaScript to create a user-friendly and responsive front-end interface that allows users to log in securely and upload crop images for analysis. Firebase Authentication is used to manage user access and ensure that only authorized users can interact with the system..

Shivam Kumar	0187CS231217
Shrit Kumar	0187CS231218
Vivek Kumar	0187CS231257
Yash Raj	0187CS231260

Guide Signature
(Dr. Komal Tahliliani)

APPENDIX-1

GLOSSARY OF TERMS

(In alphabetical order)

R

React.js React is a lightweight, component-based JavaScript library used for building interactive and dynamic user interfaces. It simplifies frontend development by enabling reusable UI components, efficient state management, and fast rendering, making it popular for modern web applications.

G

GitHub A cloud-based platform for version control and collaborative development using Git. It manages source code, tracks changes, and automates deployment workflows.

F

Firebase Firebase Firestore is a cloud-based NoSQL database management system used to store, manage, and retrieve application data in real time. It is known for its scalability, reliability, and seamless integration with web applications and cloud services.

P

Python Python is a high-level, cross-platform programming language that runs on Windows, Linux, macOS, and other systems. It is widely used for backend development and machine learning due to its simplicity, readability, and extensive library support..